Day 62: The Week 9 Build — a Many-to-Many RNN Tagger From Six Exercise Parts

MD Mutasim Billah | 52-Week Data Science to ML/AI Roadmap | Week 9, Day 7 | Complete study guide

Watch: Types of RNN | Recurrent Neural Network Types | Deep Learning Tutorial 34 (codebasics) | Part Of Speech POS Tagging: NLP Tutorial For Beginners - S1 E11 (codebasics)

Script: `day62_many_to_many_tagger.py` · every number in this guide is copied by program from `day62_run_output.txt`, the real output of that script.

Yesterday the RNN read a whole review and gave **one** answer. Today it gives an answer for **every word**: the many-to-many RNN. Today is also the last day of Week 9, so the model is not written fresh. It is **assembled from the six short exercise scripts of Days 56–61**, with one new line added. The task is part-of-speech tagging on real, hand-labelled English sentences. The tagger reaches **0.8327** word accuracy on unseen sentences against **0.8016** for a baseline with no memory, and the guide shows exactly where that gain comes from, where the RNN is *worse*, and what it still cannot see.

0. Where today fits

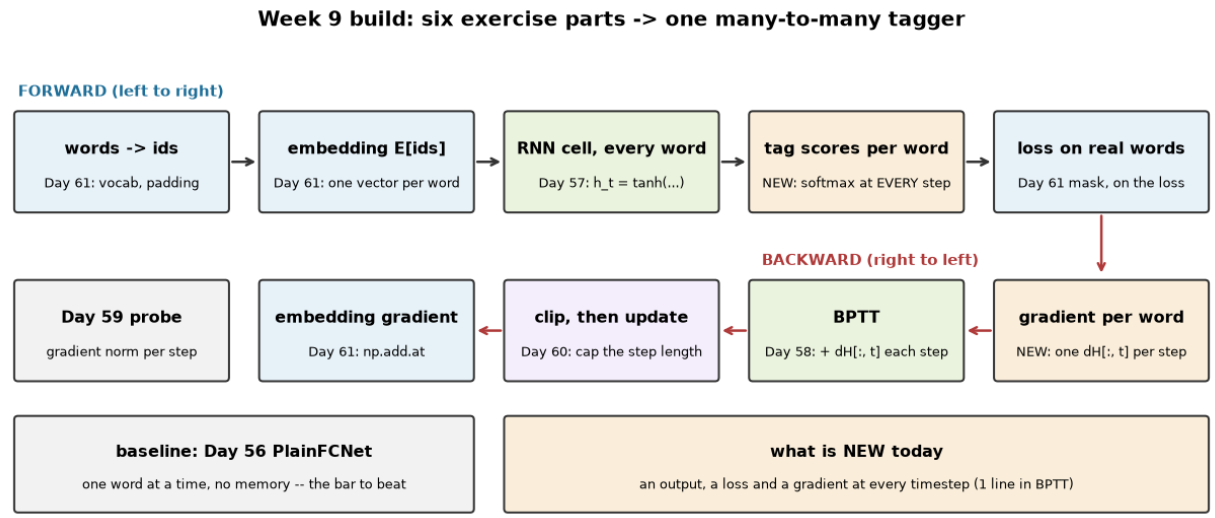
DEFINITION — start here, no prior knowledge assumed

An **integration day** is a day with almost no new ideas. Parts that were built and tested one at a time are connected into one working system, and the system is tested as a whole. The one idea that *is* new today: **many-to-many** means the network produces an output at **every** step of the sequence, not just one at the end.

REAL-WORLD SCENARIO

A bicycle factory makes wheels, a frame, brakes, a chain, gears and a saddle on six different days, and tests each one alone. On the seventh day it bolts them together and rides the bicycle. A wheel that passed its own test can still rub against the frame, so the bicycle is tested again as a whole. That seventh day is today.

SYSTEM DIAGRAM



How the six exercise parts connect. Orange boxes are the only new idea: an output, a loss and a gradient at every timestep.

ACTUAL OUTPUT

```
from              what is reused                          changed today?
Day 56 exercise   softmax, one_hot, PlainFCNet            no -- verbatim
Day 57 exercise   rnn_step_forward, rnn_forward           2 lines (a vector per word)
Day 58 exercise   rnn_backward (BPTT)                     1 new line (gradient at EVERY step)
Day 59 exercise   grad_norm_by_timestep                   the same 1 new line
Day 60 exercise   clip_by_global_norm                     no -- verbatim
Day 61 exercise   ids + padding, E[ids], np.add.at, mask  mask applied to the LOSS
```

How to read each section

Step	What it gives you
DEFINITION	What the idea is, in plain words. Start here.
REAL-WORLD SCENARIO	An everyday situation where the same idea shows up.
MATHEMATICS	The formula, when the section introduces one.
SPECIFICATION	The exact setup of the experiment: data, sizes, settings, what is measured.
Why this test	The question the experiment answers.
IMPLEMENTATION	The real code from the script that produces the result.
ACTUAL OUTPUT	What the script really printed, copied unchanged.
What it means	How to read the numbers, and what they do and do not show.
Why choose this? / Why NOT this?	When the idea is the right tool, and what it costs.

Key words used today

Word	Plain meaning
Many-to-many	Many inputs, and an output for each one: a tag for every word.
Part of speech (POS)	A word's job in the sentence: NOUN (thing), VERB (action), ADJ (describes a thing), DET (the, a), ADP (in, on, of), PRON (I, it), PROPN (a name), AUX (is, have, will), PUNCT, and so on.
Tagging	Giving every word of a sentence one label.
Ambiguous word	A word that can have more than one tag. *Book*: a thing, or the action of booking.
Unknown word, <unk>	A word the model never learned, because it was too rare in the training sentences.
Context	The other words around a word.
Hidden state / memory	The RNN's running summary of the words read so far.
Baseline	A simple method used as the bar a fancier method must beat.
Word-level accuracy	Out of all real words (padding left out), the fraction given the correct tag.

1. The task and the data: one tag for every word

DEFINITION — start here, no prior knowledge assumed

Part-of-speech tagging means labelling every word of a sentence with its grammatical job: noun, verb, adjective, and so on. It is the classic **many-to-many** task: a sentence of 9 words goes in, 9 tags come out, one per word, in the same order.

The data is the **Universal Dependencies English Web Treebank**: real sentences from blogs, emails, reviews and question-and-answer sites, where linguists wrote the correct tag under every word by hand.

REAL-WORLD SCENARIO

Think of a school grammar exercise: *underline every noun in blue and every verb in red*. The pupil gives one answer per word, not one answer per sentence. Phone keyboards do a version of this to guess your next word, and voice assistants do it to work out which word in “book a table” is the action.

It is harder than it looks because many words change jobs. In “I **can** open the **can**” the same three letters are first a helper verb and then a thing. Only the neighbouring words tell you which.

SPECIFICATION

Source	UD English Web Treebank (Silveira et al., 2014), licence CC BY-SA 4.0. A trimmed copy (word and tag columns only) is in `ud_english_ewt_pos/` with its licence and attribution.
Split	Train = the treebank's dev file (2,001 sentences). Validation = the first 430 sentences of its test file, test = the other 1,647, cut at a document boundary.
Words	Lowercased. A word gets its own id if it appears at least twice in training; id 0 = <pad>, id 1 = <unk> (same rule as Day 61).
Tags	The 17 universal part-of-speech tags.
Padding	Each split is padded to its longest sentence; a True/False mask marks the real words.
Measured	Sizes, vocabulary, unknown-word rate, how common each tag is.

Why this test

The split follows the original files, so no document appears in both training and test. Sentences from one blog post share names and topics; mixing them across the split would let the model look better than it is. The treebank's own, much larger training file is deliberately not used: this is a small “toy” version, and the unknown-word rate below is the price.

IMPLEMENTATION

```
def read_tagged(name):
    sentences, current = [], []
    with open(os.path.join(DATA_DIR, name + ".txt"), encoding="utf-8") as f:
        for line in f:
            line = line.rstrip("\n")
            if not line:
                sentences.append(current); current = []
            else:
                word, tag = line.split("\t")
                current.append((word.lower(), tag))
    return sentences
def encode(sentences, stoi, ttoi):
    T = max(len(s) for s in sentences)
    ids = np.full((len(sentences), T), PAD, dtype=int)
    tags = np.zeros((len(sentences), T), dtype=int)
    lens = np.zeros(len(sentences), dtype=int)
    for row, sent in enumerate(sentences):
        ids[row, :len(sent)] = [stoi.get(w, UNK) for w, _ in sent]
        tags[row, :len(sent)] = [ttoi[t] for _, t in sent]
        lens[row] = len(sent)
```

```
    mask = np.arange(T)[None, :] < lens[:, None]
    return ids, tags, mask
```

The subtle part: the last line builds the mask with no loop. It compares every position number with every sentence length, giving True exactly on the real words.

ACTUAL OUTPUT

```
split        sentences    words  longest
train             2001    25148       75
validation         430     6634       81
test              1647    18460       65
distinct words in training sentences: 4812; kept (seen at least twice): 2080; vocabulary size with
    <pad> and <unk>: 2082
test words that are <unk>: 20.2%
17 tags, by share of training words:
   NOUN 16.7%   PUNCT 12.2%    VERB 10.8%    PRON  8.8%     ADP  8.1%     DET  7.6%
  PROPN  7.4%     ADJ  7.4%     AUX  6.2%     ADV  4.9%   CCONJ  3.1%    PART  2.6%
  SCONJ  1.6%     NUM  1.5%    INTJ  0.5%     SYM  0.3%       X  0.2%
worked example (training sentence):
  words: the       sheikh    in        wheel     -         chair     has       been      attacked
  tags : DET       NOUN      ADP       NOUN      PUNCT     NOUN      AUX       AUX       VERB
Actual output from this lesson's run.
```

What it means

About 25,000 training words is small. Of 4812 different words in training, only 2080 appear twice or more, so **20.2% of test words are unknown** to the model — one in five. Every accuracy number below should be read with that in mind.

The tags are far from evenly spread: NOUN alone is 16.7% of words, while INTJ, SYM and X together are about 1%. A model that always answered NOUN would score about 17%, so that is the true floor, not 1/17.

WHY CHOOSE THIS?

Part-of-speech tagging is the cleanest many-to-many task: one label per word, a clear right answer, and position matters, so (unlike Day 61's sentiment task) a model that ignores word order cannot do well.

WHY NOT THIS? (TRADEOFFS)

With only 2,001 training sentences one test word in five is unknown, which caps what any model can reach here. Lowercasing also throws away capital letters, a strong hint that a word is a name.

2. The assembled model, and the one new line

DEFINITION — start here, no prior knowledge assumed

A **many-to-many RNN** reads the words one at a time, like Day 61's network, but after **every** word it also makes a prediction from its current hidden state. So there is one prediction, one piece of loss and one gradient per word. The total loss is the **average over all real words**.

In the backward pass this adds exactly one thing: at every timestep, the gradient from that word's own prediction joins the gradient flowing back from later words.

REAL-WORLD SCENARIO

Many-to-one (Day 61) is a teacher who reads a whole essay and writes one mark at the bottom. **Many-to-many** is a teacher who corrects a dictation: a tick or a cross under every single word. The pupil learns much faster from the second kind, because each mistake is pointed at directly.

Live subtitles work the same way: the system cannot wait for the speaker to finish the whole speech; it must output something for each word as it arrives.

MATHEMATICS

$h_t = \tanh(x_t W_{xh} + h_{t-1} W_{hh} + b_h)$ *the Day 57 cell, unchanged*

$p_t = \text{softmax}(h_t W_{hy} + b_y)$ *NEW: a tag distribution at* ***every*** *t, not only the last*

$L = -(1/N) \Sigma_{\text{real words}} \log p_t[\text{true tag}_t]$ *N = number of real words; padding contributes 0*

$dh_t = (p_t - y_t) W_{hy}^T / N + da_{t+1} W_{hh}^T$ *own output's gradient + what flows back from the future*

$da_t = dh_t \cdot (1 - h_t^2)$ *then accumulate* dW_{xh}, dW_{hh}, db_h *exactly as on Day 58*

SPECIFICATION

Network under test	A tiny RNNTagger: 12 words, 3 tags, embedding 4, hidden 5, no clipping.
Batch	4 sentences of lengths 5, 2, 4, 1 (so padding and the mask are exercised), with one word appearing twice in the same sentence.
Check	Every entry of all six parameters (E, Wxh, Whh, bh, Why, by) is nudged by ±1e-5; the change in the masked loss is compared with the gradient from the assembled backward pass.
Pass rule	|numeric – analytic| ≤ 1e-10 + 1e-6 × |numeric| for every entry, and the <pad> row of the embedding gradient exactly zero.
Follow-up	The single entry with the largest relative error is re-measured at three nudge sizes.

Why this test

Each part passed its own check on its own day. That does not prove they work **together**: the shapes changed (a vector per word instead of one number), the loss changed (averaged over words, with a mask), and a line was added to BPTT. So the check is run through the whole assembled pipeline — words in, loss out — using the same loss the real training uses.

IMPLEMENTATION — Day 58's BPTT with the one new line

```
def rnn_backward(dH, X, H, h0, Wxh, Whh):
    m, T, _ = X.shape
    dWxh = np.zeros_like(Wxh)
    dWhh = np.zeros_like(Whh)
    dbh = np.zeros(Whh.shape[0])
    dX = np.zeros_like(X)
    dh = np.zeros_like(h0)                      # <-- CHANGED
    for t in reversed(range(T)):
        dh = dh + dH[:, t, :]                   # <-- NEW TODAY
        h_prev = H[:, t - 1, :] if t > 0 else h0
        da = dh * (1.0 - H[:, t, :] ** 2)
        dWxh += X[:, t, :].T @ da
        dWhh += h_prev.T @ da
        dbh += da.sum(axis=0)
        dX[:, t, :] = da @ Wxh.T
        dh = da @ Whh.T
    return dWxh, dWhh, dbh, dX
```

The subtle part: `dh = dh + dH[:, t, :]` is the whole of many-to-many in the backward pass. If dH is zero everywhere except the last step, this function is Day 58's again.

IMPLEMENTATION — the assembled tagger: forward, loss, gradients

```
def forward(self, ids):
    m, T = ids.shape
    self.ids = ids
    self.h0 = np.zeros((m, self.hidden_dim))
    self.X = self.E[ids]
    self.H = rnn_forward(self.X, self.h0, self.Wxh, self.Whh, self.bh)
    self.H2 = self.H.reshape(m * T, self.hidden_dim)
```

```
    self.probs = softmax(self.H2 @ self.Why + self.by)
    return self.probs
def loss(self, tags, mask):
    p_true = self.probs[np.arange(tags.size), tags.ravel()]
    return -np.sum(np.log(p_true + 1e-9) * mask.ravel()) / mask.sum()
def gradients(self, tags, mask):
    m, T = self.ids.shape
    dlogits = (self.probs - one_hot(tags.ravel(), self.n_tags)) * mask.ravel()[:, None] /
        mask.sum()
    grads = dict(Why=self.H2.T @ dlogits, by=dlogits.sum(axis=0))
    dH = (dlogits @ self.Why.T).reshape(m, T, self.hidden_dim)
    grads["Wxh"], grads["Whh"], grads["bh"], dX = rnn_backward(dH, self.X, self.H, self.h0,
        self.Wxh, self.Whh)
    grads["E"] = np.zeros_like(self.E)
    np.add.at(grads["E"], self.ids, dX)
    if not self.memory:
        grads["Whh"] = np.zeros_like(self.Whh)
    return grads
```

The subtle part: `reshape(m * T, hidden)` lays every word of every sentence out as its own row, so Day 56's softmax and one_hot work unchanged. Multiplying by the mask sets the gradient of every padding row to zero.

ACTUAL OUTPUT

```
 parameter  entries  max relative error  max |difference|  all entries pass
         E       48            1.92e-07          2.45e-10             True
       Wxh       20            5.30e-08          5.75e-11             True
       Whh       25            3.12e-06          4.01e-11             True
        bh        5            5.49e-09          1.83e-10             True
       Why       15            1.37e-08          7.76e-11             True
        by        3            7.30e-09          1.63e-10             True
every parameter passes: True;  gradient row of <pad> exactly zero: True
Actual output from this lesson's run.
```

ACTUAL OUTPUT

```
the largest relative error is at Whh(0, 1), where the gradient itself is tiny: -4.217e-07. The
    same entry at three nudge sizes:
   nudge         numeric        analytic   |difference|
   1e-04   -4.216716e-07   -4.216712e-07       4.12e-13
   1e-05   -4.216738e-07   -4.216712e-07       2.63e-12
   1e-06   -4.217737e-07   -4.216712e-07       1.03e-10
Actual output from this lesson's run.
```

What it means

The assembled model is correct. All six parameters pass, and the padding row of the embedding table receives exactly zero gradient.

One number needed a second look. The largest relative error, 3.12e-06 on one entry of Whh, is above the 1e-6 level the earlier days stayed under. A first version of this check simply printed FAIL. Before trusting or dismissing that, the entry was examined: its gradient is tiny (-4.217e-07), and the actual disagreement is 2.63e-12. Re-measuring with a larger nudge shrinks the disagreement (4.12e-13); a smaller nudge grows it (1.03e-10). That pattern is the fingerprint of **rounding error in the numeric estimate**: a real bug in the code would give the same error whatever the nudge size. The pass rule therefore allows a tiny absolute difference as well as a relative one.

WHY CHOOSE THIS?

An output at every step gives the network far more feedback per sentence (one correction per word), and Section 6 shows it also keeps gradient alive at every position.

WHY NOT THIS? (TRADEOFFS)

It needs a label for every word, which is expensive: someone had to tag all 50,000 words here by hand. And each prediction can only use the words read so far (Section 7).

3. Two baselines with no memory

DEFINITION — start here, no prior knowledge assumed

A **baseline** is a simple method built first, so there is a bar the real model must clear. Both baselines here look at **one word at a time** and nothing else. **Most-frequent-tag** involves no learning: count, for each word, which tag it had most often in training, and always answer that. The second is **Day 56's dense network**, fed one word at a time.

A **one-hot vector** is how a word is shown to that network: a row of 2,082 numbers, all 0 except a single 1 in the position of that word.

REAL-WORLD SCENARIO

A small pocket dictionary lists only the commonest meaning of each word. Look up *book* and it says “a thing you read”, every time, even in “please book a table”. It is right most of the time, it is very cheap, and it can never get better by looking at the rest of the sentence. For a word that is not in the dictionary at all it can only make the same blind guess each time.

MATHEMATICS

$$\textit{one-hot(word i)} \times W_1 = \textit{row i of } W_1$$

multiplying by a one-hot vector just picks a row — the same thing Day 61's embedding lookup E[ids] does directly

SPECIFICATION

(a) Most-frequent-tag	For each vocabulary word, the tag it had most often in training. Unknown words get the most common tag among training words that became <unk>.
(b) PlainFCNet	Day 56's class, code unchanged: 2,082 one-hot inputs → 32 ReLU units → 17 tags. Learning rate 1.0, 20 epochs, batches of 128 words, seed 7. Best validation epoch kept.
What both lack	Any view of the other words. The same word always gets the same tag.
Measured	Word-level accuracy on validation and test.

Why this test

These baselines have everything the RNN has — same words, same vocabulary, same tags — except memory. So whatever the RNN scores above them is what reading in sequence adds. Using two keeps the comparison fair: one is pure counting, the other is a trained network, and if they agree, the bar is real.

IMPLEMENTATION

```
seen = defaultdict(Counter)
for w, t in zip(ids_tr[mask_tr], tags_tr[mask_tr]):
    seen[w][t] += 1
mft = np.full(V, seen[UNK].most_common(1)[0][0])
for w, c in seen.items():
    mft[w] = c.most_common(1)[0][0]
# (b) Day 56's network, one word at a time; this loop runs once per epoch
fc = PlainFCNet(V, hidden=32, n_classes=K, lr=1.0, seed=7)
```

```
for start in range(0, len(perm), 128):
    idx = perm[start:start + 128]
    fc.forward(one_hot_words(words[idx]))
    fc.backward(one_hot(labels[idx], K))
```

The subtle part: `seen[w][t] += 1` works on a word never seen before because defaultdict creates an empty Counter the first time a key is used.

ACTUAL OUTPUT

```
(a) most-frequent-tag: unknown words are tagged NOUN; validation 0.7673, test 0.8016
(b) PlainFCNet on one word at a time (2082 inputs -> 32 -> 17):
 epoch  train acc  val acc  test acc
     0     0.6281   0.5841    0.6292
     3     0.6385   0.6168    0.6575
     6     0.8509   0.7576    0.7998
     9     0.8526   0.7653    0.7953
    12     0.8455   0.7560    0.7938
    15     0.8443   0.7563    0.7920
    18     0.8514   0.7626    0.7960
    19     0.8495   0.7624    0.7940
best validation 0.7665 at epoch 16; test there 0.7987
tag given to EVERY unknown word: most-frequent-tag -> NOUN, PlainFCNet -> NOUN
Actual output from this lesson's run.
```

What it means

Counting scores **0.8016** on test. The trained dense network lands in the same place, 0.7987: with no view of the neighbours, learning cannot beat counting. Both give **every** unknown word the tag NOUN. So roughly 0.80 is what you get from the word alone, and it is a strong bar: four words in five are tagged correctly without reading the sentence at all.

WHY CHOOSE THIS?

Most-frequent-tag takes a few lines, no training, and is right four times in five. Always build it first; without it, 0.83 from an RNN would look far more impressive than it is.

WHY NOT THIS? (TRADEOFFS)

It cannot separate the two meanings of an ambiguous word and gives every unknown word the same tag. No amount of extra training fixes that, because the information is not in its input.

4. Training the tagger

DEFINITION — start here, no prior knowledge assumed

Training repeats one loop: tag a batch of sentences, measure how wrong the tags are (the loss), compute gradients, take a small step. An **epoch** is one pass over all training sentences. **Word-level accuracy** is the fraction of real words given the correct tag.

The **validation set** decides which epoch's network to keep; the **test set** gives the final score and is never used to choose anything.

REAL-WORLD SCENARIO

A new proofreader practises on 2,001 marked-up sentences. After each pass through them, a supervisor checks them on 430 sentences they did not practise on (validation). The version of the proofreader that did best on that check is the one sent to do the real job on 1,647 fresh sentences (test). Checking after every pass matters: Day 60 showed a learner can get good and then be ruined later.

SPECIFICATION

Network	Embedding 32 → RNN, hidden 32 (tanh) → 17-tag softmax at every word.
Training	SGD, learning rate 1.0, 25 epochs, batches of 32 sentences, gradient clipping at global norm 1.0 (Day 60). Each batch is padded only to its own longest sentence.
Repeats	Three seeds (0, 1, 2); seed 0 is shown epoch by epoch.
Tracked every epoch	Mean training loss; train, validation and test word accuracy; fraction of steps clipped; the largest gradient norm.
How settings were chosen	Hidden size 32 or 64 and learning rates from 0.5 to 4.0 were tried in exploration; this setting did best on validation. Section 6 revisits the others.

Why this test

This is the first time the assembled model learns anything. Three seeds show whether the result depends on a lucky start; the epoch-by-epoch record shows whether it learns steadily, overfits, or collapses.

IMPLEMENTATION

```
def train_step(self, ids, tags, mask):
    self.forward(ids)
    loss = self.loss(tags, mask)
    grads = self.gradients(tags, mask)
    if self.clip is None:
        norm = np.sqrt(sum(np.sum(g ** 2) for g in grads.values()))
    else:
        grads, norm = clip_by_global_norm(grads, self.clip)
    for name, g in grads.items():
        getattr(self, name)[...] -= self.lr * g
    return loss, norm, self.clip is not None and norm > self.clip
# the training loop inside fit()
for epoch in range(epochs):
    perm = rng.permutation(len(ids_tr))
    losses, norms, clips = [], [], []
    for start in range(0, len(perm), batch_size):
        idx = perm[start:start + batch_size]
        T_batch = lens_tr[idx].max()
        loss, norm, clipped = net.train_step(ids_tr[idx, :T_batch], tags_tr[idx, :T_batch],
            mask_tr[idx, :T_batch])
        losses.append(loss); norms.append(norm); clips.append(clipped)
    hist["loss"].append(np.mean(losses)); hist["clipped"].append(np.mean(clips));
        hist["max_norm"].append(np.max(norms))
    hist["train"].append(token_accuracy(net.predict(ids_tr), tags_tr, mask_tr))
    hist["val"].append(token_accuracy(net.predict(ids_va), tags_va, mask_va))
    hist["test"].append(token_accuracy(net.predict(ids_te), tags_te, mask_te))
    if hist["val"][-1] > best_val:
        best_val, best_net, best_epoch = hist["val"][-1], copy.deepcopy(net), epoch
```

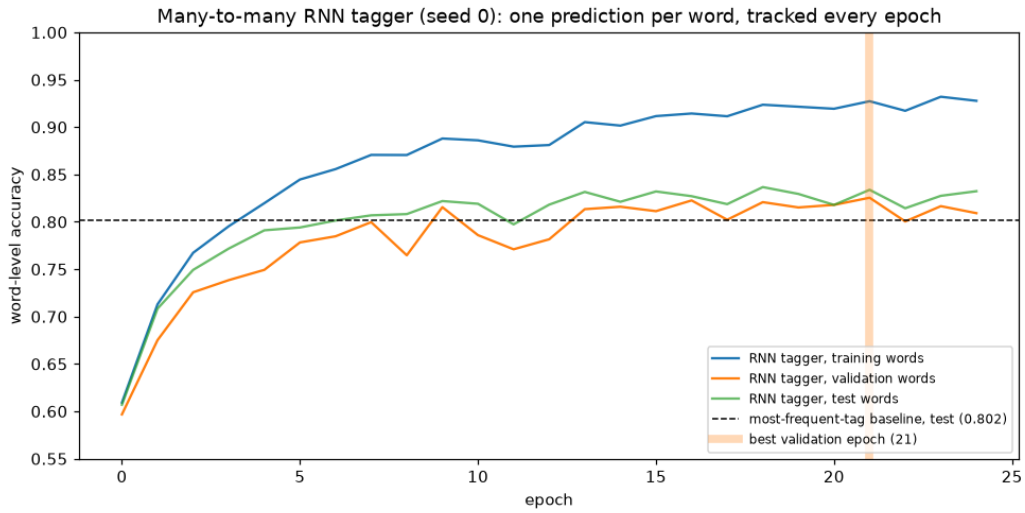
The subtle part: the gradient norm is measured even when clipping is off, so Section 6 can compare the largest gradient with and without it.

ACTUAL OUTPUT

```
seed 0:
 epoch  train loss  train acc  val acc  test acc  steps clipped  largest grad
     0      1.8593     0.6092   0.5968    0.6069           0.0%         0.560
     3      0.6930     0.7954   0.7385    0.7718           0.0%         0.608
     6      0.4904     0.8557   0.7847    0.8014           0.0%         0.709
     9      0.3839     0.8879   0.8155    0.8219           0.0%         0.772
    12      0.3231     0.8811   0.7816    0.8181           0.0%         0.605
    15      0.2971     0.9116   0.8113    0.8321           0.0%         0.820
    18      0.2650     0.9236   0.8208    0.8367           0.0%         0.749
    21      0.2425     0.9273   0.8254    0.8338           0.0%         0.568
    24      0.2244     0.9277   0.8092    0.8323           0.0%         0.622
Actual output from this lesson's run.
```

ACTUAL OUTPUT

```
 seed  best epoch  val there  test there  train at end  test at end
    0          21     0.8254      0.8338        0.9277       0.8323
    1          12     0.8253      0.8334        0.9324       0.8236
    2          17     0.8274      0.8308        0.9277       0.8167
RNN tagger test accuracy, mean of 3 seeds: 0.8327  (min 0.8308, max 0.8338)
no-memory baselines on the same test words: most-frequent-tag 0.8016, PlainFCNet 0.7987
test set: 18460 words, so one standard error is about 0.003
Actual output from this lesson's run.
```



Seed 0. Training accuracy (blue) keeps rising; validation (orange) and test (green) level off just above the most-frequent-tag baseline (dashed).

What it means

It learns, and it beats the baseline. Over three seeds the tagger averages **0.8327** on test (from 0.8308 to 0.8338) against 0.8016 for most-frequent-tag. The gap is about 3 points. With 18460 test words one standard error is about 0.003, so the gap is roughly ten standard errors: it is real. It is also modest.

It overfits mildly. By the last epoch training accuracy is about 0.9277 while test stays near 0.83. Validation accuracy wobbles from epoch to epoch, which is why the best epoch differs by seed (21, 12, 17).

Clipping almost never fires here. The largest gradient norm in the seed-0 table stays below the threshold of 1.0. Section 6 looks at when clipping does matter.

WHY CHOOSE THIS?

One short training run (well under a minute on a laptop) gives a tagger that is measurably better than counting, from parts that were each written and checked in a few lines.

WHY NOT THIS? (TRADEOFFS)

A 3-point gain over a no-learning baseline is small. The model is limited by 2,001 training sentences and by the plain RNN cell, not by training time: more epochs only overfit.

5. Where does memory help?

DEFINITION — start here, no prior knowledge assumed

Memory here means the hidden state: the RNN's running summary of the words before the current one. A word's **context** is the words around it. An **ablation** removes one thing on purpose to see what it was doing; here the thing removed is the recurrent connection (memory off), or the word order (words shuffled).

Test words are sorted into three groups: **unambiguous** (a known word that had one tag in training), **ambiguous** (a known word that had several), **unknown** (<unk>).

REAL-WORLD SCENARIO

You meet a word you have never seen: “She decided to **flurb** the report.” You do not know what flurb means, but you know it is an action, because it comes right after “decided to”. In “the **flurb** was on the table” you would say it is a thing. You are using the words before it. A pocket dictionary cannot do that; a reader with memory can.

The opposite also happens. If you over-trust the pattern “a capital-looking word at the start of a sentence is a name”, you will sometimes call an ordinary word a name.

SPECIFICATION

Models compared	Most-frequent-tag; PlainFCNet; the RNN tagger with memory **off** (recurrent weights held at exactly zero during training, everything else identical); the RNN tagger. Trained models: mean of three seeds.
Word groups	Unambiguous / ambiguous / unknown, as defined above, plus all words.
Unknown words by context	Unknown test words grouped by the word just before them. For each group: its most common true tag, the tag the RNN picks most, and both accuracies.
Shuffle test	Every test sentence's words put in one fixed random order (each word keeps its true tag); the trained tagger is evaluated again.
Examples	The short test sentences (5–9 words) where the seed-0 tagger gains most, and loses most, against most-frequent-tag. Chosen by program, not by hand.

Why this test

A 3-point gain overall says little about *how* it was won. If memory is really being used, the gain should be concentrated in the words that need context (ambiguous and unknown ones), it should vanish when the recurrent link is switched off, and it should be damaged by shuffling the words. On Day 61 the shuffle test showed the RNN was **not** using order. The same test is repeated here on purpose.

IMPLEMENTATION

```
groups = {"all words": mask_te,
          "unambiguous": mask_te & (ids_te != UNK) & ~base["ambiguous"][ids_te],
          "ambiguous": mask_te & base["ambiguous"][ids_te],
          "unknown": mask_te & (ids_te == UNK)}
# the shuffle test: one fixed random order per test sentence
order = np.tile(np.arange(ids_te.shape[1]), (len(ids_te), 1))
for row, n in enumerate(mask_te.sum(axis=1)):
    order[row, :n] = rng.permutation(n)
ids_sh = np.take_along_axis(ids_te, order, axis=1)
tags_sh = np.take_along_axis(tags_te, order, axis=1)
# unknown words, grouped by the word before them
prev = np.full_like(ids_te, -1)
prev[:, 1:] = ids_te[:, :-1]
    sel = groups["unknown"] & np.isin(prev, before_ids)
```

The subtle part: memory is switched off by multiplying the starting Whh by 0 and zeroing its gradient every step (in RNNTagger), so that network is the same code, same sizes, same training — only the link from one word to the next is missing.

ACTUAL OUTPUT

```
                          all words   unambiguous     ambiguous       unknown
share of test words          100.0%         37.7%         42.2%         20.2%
most-frequent-tag            0.8016        0.9814        0.8474        0.3705
PlainFCNet (Day 56)          0.7987        0.9814        0.8404        0.3705
RNN, memory off              0.7714        0.9810        0.8367        0.2440
RNN tagger                   0.8327        0.9693        0.8740        0.4914
tag the memory-off tagger (seed 0) gives unknown words: PROPN (100% of them)
Actual output from this lesson's run.
```

ACTUAL OUTPUT

```
UNKNOWN test words, grouped by the word just before them (accuracy = mean of 3 seeds):
word before           count  most common true tag  RNN mostly says  RNN acc  most-freq acc
(start of sentence)     359           PROPN (41%)     PROPN (100%)    0.412          0.162
the / a / an            445            NOUN (62%)      NOUN (100%)    0.618          0.625
to                       95            VERB (66%)       VERB (96%)    0.639          0.063
i / you / we / they      52            VERB (77%)      VERB (100%)    0.769          0.058
of / in / for / on      205           PROPN (45%)       NOUN (74%)    0.434          0.307
,                       100            NOUN (46%)      PROPN (44%)    0.303          0.460
another unknown word    715            NOUN (45%)       NOUN (62%)    0.445          0.450
Actual output from this lesson's run.
```

ACTUAL OUTPUT

```
shuffle test -- RNN tagger on normal test sentences 0.8327, on the same sentences with words
    shuffled 0.7487  (most-frequent-tag is 0.8016 either way)
Actual output from this lesson's run.
```

ACTUAL OUTPUT

```
a short test sentence where memory helps most:
  word      atal     pharmacy ,        karol    bag      .
  in vocab? NO       NO       yes      NO       NO       yes
  true tag  PROPN    PROPN    PUNCT    PROPN    PROPN    PUNCT
  most-freq NOUN     NOUN     PUNCT    NOUN     NOUN     PUNCT
  RNN       PROPN    PROPN    PUNCT    PROPN    PROPN    PUNCT
a short test sentence where the RNN does worse:
  word      mountain ,        cliffs   ,        peat     bogs     ,        valleys
  in vocab? NO       yes      NO       yes      NO       NO       yes      NO
  true tag  NOUN     PUNCT    NOUN     PUNCT    NOUN     NOUN     PUNCT    NOUN
  most-freq NOUN     PUNCT    NOUN     PUNCT    NOUN     NOUN     PUNCT    NOUN
  RNN       PROPN    PUNCT    PROPN    PUNCT    NOUN     PROPN    PUNCT    PROPN
Actual output from this lesson's run.
```

What it means

The gain is where context is needed. On unknown words the tagger scores 0.4914 against 0.3705 for the no-memory baselines; on ambiguous words 0.8740 against 0.8474.

Memory has a cost. On unambiguous words — the easy ones — the tagger is slightly *worse*: 0.9693 against 0.9814. A model that listens to context will sometimes let context overrule a word whose tag was never in doubt.

The gain disappears with memory off. The identical tagger without its recurrent link scores 0.7714, *below* counting. Its unambiguous and ambiguous numbers match the baselines; the difference is that it settled on PROPN for every unknown word (0.2440 correct) where counting picks NOUN (0.3705). With no context it must give one answer for all unknown words, and it picked the less common one.

The unknown-word table shows the mechanism. After *to*, the tagger calls an unknown word a VERB 96% of the time and is right 0.639 of the time, against 0.063 for the baseline. After *i / you / we / they*: 0.769 against 0.058. At the start of a sentence it always says PROPN (0.412 against 0.162). It is reading the previous word, as a person would.

And where it fails. After *the / a / an* both say NOUN, so there is no gain. After a comma the tagger is *worse* (0.303 against 0.460). When the previous word is itself unknown — the largest group, 715 words — there is nothing to go on and no gain (0.445 against 0.450).

Order matters now. Shuffling the test words drops the tagger from 0.8327 to 0.7487, below the baseline. On Day 61 the same test changed nothing. This model uses order; that one did not.

The two example sentences are both lists of unknown words. In the first they are place names and the tagger's habit of calling unknown words names is right four times. In the second they are ordinary nouns and the same habit is wrong four times. One habit, both outcomes.

WHY CHOOSE THIS?

Memory lets the model tag words it has never seen from the words before them, and pick between the meanings of an ambiguous word. Those are exactly the cases a dictionary-style model cannot touch.

WHY NOT THIS? (TRADEOFFS)

It makes a few new mistakes on easy words, it leans on shallow habits (sentence start means name), and when the context is itself unknown it has nothing to use.

6. Two parts at work: Day 59's probe and Day 60's clipping

DEFINITION — start here, no prior knowledge assumed

The **gradient norm at a word's hidden state** measures how strongly the loss can still influence what the network did at that word. If it is near zero, that word's processing cannot be corrected. Day 59's **probe** records this number at every position.

Gradient clipping (Day 60) shortens any update step whose gradient is longer than a threshold, without changing its direction.

REAL-WORLD SCENARIO

The probe. Two classes do the same 20-question worksheet. In one, the teacher marks every question straight away. In the other, the teacher only says at the end whether the last answer was right. In the second class nobody learns anything about question 1: the feedback is too far away and too faint by the time it gets there. Feedback at every step keeps every step teachable.

Clipping. A fuse box does nothing for 364 days a year. On the one day a faulty kettle draws a huge current, it trips and saves the house. Measuring “how often it trips” badly undersells it.

MATHEMATICS

Day 59's recursion, with today's one change: $dh_t = dH[:, t] + (dh_{t+1} \cdot (1 - h_{t+1}^2)) W_{hh}^T$

Many-to-one (Day 61): dH is zero except at the last word, so dh_0 *must travel through all T steps.*

Many-to-many (today): dH[:, t] is non-zero at every word, so each dh_t *has a term that travelled zero steps.*

SPECIFICATION

(a) Probe	The trained seed-0 tagger and all 26 test sentences of exactly 20 words (no padding). Gradient norm at each word's hidden state, for two losses on the same network: today's (a loss at every word) and a loss on the last word only (Day 61's shape). “Own output only” is the part of today's gradient that comes straight from that word's own prediction.
(b) Clipping	The tagger trained with clipping at 1.0 and with no clipping, at four settings: hidden 32 with learning rate 1.0 (this lesson's setting) and 2.0; hidden 64 with learning rate 1.5 and 2.0. Three seeds each, 24 runs.
Reported for (b)	Test accuracy at the best validation epoch per seed; fraction of steps that were clipped; the largest gradient norm seen in any step.

Why this test

Days 59 and 60 were about failures: gradient that fades over distance, and gradient that explodes. The week's last question is whether those problems are still present in the finished model. The probe answers the first by measurement. For clipping, the honest test is to switch it off and see what breaks — not only at the setting chosen for the lesson, where things are calm, but at nearby settings too.

IMPLEMENTATION

```
def grad_norm_by_timestep(dH, H, Whh):
    T = H.shape[1]
    norms = np.zeros(T)
    dh = np.zeros_like(H[:, 0, :])              # <-- CHANGED
    for t in reversed(range(T)):
        dh = dh + dH[:, t, :]
        norms[t] = np.linalg.norm(dh)
        dh = (dh * (1.0 - H[:, t, :] ** 2)) @ Whh.T
    return norms
# the two losses compared on the same trained network
dH_every = (dlogits @ net.Why.T).reshape(len(rows), 20, -1)
dH_last = np.zeros_like(dH_every); dH_last[:, -1, :] = dH_every[:, -1, :]
n_every = grad_norm_by_timestep(dH_every, net.H, net.Whh)
n_last = grad_norm_by_timestep(dH_last, net.H, net.Whh)
```

The subtle part: `dH_last` keeps only the last word's gradient and zeroes the rest. Fed to the same probe, it reproduces Day 59's situation exactly.

ACTUAL OUTPUT

```
(a) 26 test sentences of exactly 20 words; gradient norm at each word's hidden state:
 word position  loss on every word  own output only  loss on last word only
             0           3.07e-02         1.51e-02                1.05e-05
             4           2.88e-02         1.77e-02                5.84e-05
             9           2.93e-02         2.01e-02                2.00e-04
            14           3.76e-02         2.67e-02                1.29e-03
            17           3.48e-02         2.61e-02                5.45e-03
            18           2.77e-02         2.59e-02                9.82e-03
            19           1.23e-02         1.23e-02                1.23e-02
ratio first word / last word:  every-word loss 2.49,  last-word-only loss 8.48e-04
Actual output from this lesson's run.
```

ACTUAL OUTPUT

```
(b) test accuracy at the best validation epoch, three seeds each:
 hidden   lr  clip   seed 0   seed 1   seed 2     mean  steps clipped  largest grad
     32  1.0   1.0   0.8338   0.8334   0.8308   0.8327          0.06%          1.20
     32  1.0  None   0.8338   0.8231   0.8311   0.8294          0.00%          1.20
     32  2.0   1.0   0.8287   0.8320   0.8287   0.8298          0.13%          1.36
     32  2.0  None   0.8287   0.8330   0.8300   0.8306          0.00%          1.34
     64  1.5   1.0   0.8236   0.8227   0.8228   0.8230          0.23%          3.19
     64  1.5  None   0.8232   0.5139   0.8224   0.7198          0.00%        595.92
     64  2.0   1.0   0.8264   0.8146   0.8208   0.8206          0.61%          4.73
     64  2.0  None   0.4859   0.5402   0.4530   0.4930          0.00%       5541.60
```

```
Actual output from this lesson's run.
```



Left: accuracy by word group. Middle: gradient reaching each word's hidden state (log scale). Right: test accuracy with and without clipping; each dot is one seed.

What it means

(a) A loss at every word keeps every word teachable. With today's loss, the gradient at the first word is 3.07e-02, as large as anywhere else in the sentence. With a loss on the last word only, the same network passes back 1.05e-05 to the first word: 8.48e-04 of what the last word gets. The vanishing is still there in the recurrent path — Day 59's finding holds for this trained network. Many-to-many does not cure it; it **avoids depending on it**, because every word also gets gradient straight from its own prediction: at the first word that direct part alone is 1.51e-02, about half the size of the total 3.07e-02.

The catch: gradient for *long-range* links still has to travel the fading path. The tagger can learn “the word before me was *to*” easily; something that depends on a word 15 positions back would have to be learned through that faded path (not tested here). Tagging mostly needs nearby words.

(b) At this lesson's setting, clipping does nothing measurable. Hidden 32: clipped 0.8327, unclipped 0.8294; at double the learning rate 0.8298 and 0.8306. Clipping fired on at most 0.13% of steps and the largest gradient was 1.36. No collapse either way.

At hidden 64 it is the difference between working and not. At learning rate 1.5, one unclipped seed out of three collapsed to 0.5139 after a gradient of 595.92. At learning rate 2.0, **all three** unclipped seeds collapsed (mean 0.4930; largest gradient 5541.60), while all three clipped runs finished near 0.82. In those rescued runs clipping fired on only 0.61% of steps.

So the fuse-box picture is accurate: clipping is idle almost all the time, and the few steps it does catch decide whether the run survives. An earlier draft of this lesson tested only hidden 32 and would have concluded “clipping is unnecessary for this task”. The exploration runs at hidden 64 are why that sentence is not here.

WHY CHOOSE THIS?

The probe costs one extra line and tells you whether early positions can learn at all. Clipping costs one line and, in the settings where it matters, is the difference between 0.82 and 0.49.

WHY NOT THIS? (TRADEOFFS)

Clipping adds a threshold to choose, and on calm settings it changes nothing, which can make it look useless. The probe measures gradient size, not whether the right thing is being learned.

7. A limit: this RNN reads left to right only

DEFINITION — start here, no prior knowledge assumed

A **unidirectional** (one-direction) RNN builds its hidden state from the words before the current one. When it tags word 5 it has seen words 1 to 5 and **nothing after**. A **bidirectional** RNN runs two RNNs, one in each direction, and lets each prediction use both. Only the one-direction kind is built this week.

REAL-WORLD SCENARIO

Cover a sentence with a card and slide it right, one word at a time, naming each word's job as it appears: “I saw her **duck**…” Is *duck* a bird she owns, or something she did? You cannot know until you see what follows: “…under the table” (an action) or “…swim away” (a bird). A left-to-right reader must answer before it gets there.

SPECIFICATION

Experiment	Every sentence in train, validation and test is written backwards, and the **same** tagger (same code, same settings) is trained on them. That model reads right to left: at each word it has seen only the words that *follow* it.
Comparison	Its test predictions are flipped back into normal order and compared word by word with the left-to-right tagger's. Three seeds, paired by seed.
Reported	Accuracy of each; how often both are right; how often at least one is right; how often they disagree; accuracy by word group.
Also	The six tags the left-to-right tagger (seed 0) gets wrong most often.

Why this test

The first plan for this section was to show that the *first* word of a sentence, having no words before it, gains least from the RNN. The measurement said the opposite: the first word gained **most**, because “I am at the start of a sentence” is itself useful information (Section 5's table). So that plan was dropped and replaced by a direct test: if the words to the right carry information the left-to-right reader lacks, a right-to-left reader should get **different** words right.

IMPLEMENTATION

```
def reverse_sentences(ids, tags, mask):
    ids_r, tags_r = ids.copy(), tags.copy()
    for row, n in enumerate(mask.sum(axis=1)):
        ids_r[row, :n] = ids[row, :n][::-1]
        tags_r[row, :n] = tags[row, :n][::-1]
    return ids_r, tags_r, mask
# inside demo_limits(): train on reversed data, then flip the predictions back
data_rev = [reverse_sentences(*split) for split in data]
p_rev = fit_cached(data_rev, V, K, seed=seed, name="right-to-left", lr=1.0,
    clip=1.0)[0].predict(data_rev[2][0])
p_b = p_rev.copy()
for row, n in enumerate(n_real):
    p_b[row, :n] = p_rev[row, :n][::-1]
```

The subtle part: only the first n positions of each row are reversed, so the padding stays at the end and the same mask is still correct.

ACTUAL OUTPUT

```
 seed  left-to-right  right-to-left  both right  at least one right  they disagree
    0         0.8338         0.8353      0.7671              0.9020         0.1788
    1         0.8334         0.8314      0.7618              0.9029         0.1812
    2         0.8308         0.8306      0.7575              0.9039         0.1891
Actual output from this lesson's run.
```

ACTUAL OUTPUT

```
word group      left-to-right  right-to-left
unambiguous            0.9693         0.9734
ambiguous              0.8740         0.8905
unknown                0.4914         0.4478
Actual output from this lesson's run.
```

ACTUAL OUTPUT

```
the tags the left-to-right tagger (seed 0) gets wrong most often:
 true tag  test words  accuracy   most common wrong answer
        X          40    0.0000         NOUN (55% of them)
      NUM         376    0.4335        PROPN (27% of them)
    PROPN        1306    0.5291         NOUN (37% of them)
     INTJ          96    0.5417        PROPN (23% of them)
    SCONJ         250    0.6080          ADP (31% of them)
      SYM          87    0.6092        PUNCT (17% of them)
Actual output from this lesson's run.
```

What it means

Same score, different mistakes. The two readers are equally accurate (seed 0: 0.8338 and 0.8353), yet they disagree on about 18% of words (0.1788), and on about 90% of words (0.9020) at least one of them is right.

That 0.90 is **not** a score any model here achieved: it assumes you magically know which reader to trust for each word. What it shows is that the words to the right contain information the left-to-right tagger never gets, and the other way round. That is the reason bidirectional RNNs exist.

Each direction is better at something. Right-to-left does better on ambiguous words (0.8905 against 0.8740): what follows a word is a good clue to which of its meanings is meant. Left-to-right does better on unknown words (0.4914 against 0.4478).

The weakest tags. PROPN (names) is right only 0.5291 of the time, mostly mistaken for NOUN; NUM (numbers) 0.4335. Both are mostly unknown words. Two likely causes that were **not tested here**: words were lowercased, which removes the capital letter that marks a name, and numbers are treated as ordinary rare words. X (“other”, 40 test words) is never right.

WHY CHOOSE THIS?

A one-direction RNN can tag words as they arrive, which is what live uses (typing, speech) need, and it is the simplest thing that uses context at all.

WHY NOT THIS? (TRADEOFFS)

It is blind to everything after the current word. When the whole sentence is available, as it is here, that throws away information for no reason; a bidirectional model is the standard fix.

8. Week 9 in one table

Every row's number is the real result reported on that day (see the week README for each day's details).

Day	Part built	What the day's experiment showed
56	The problem	A dense net learned an order task (0.888) and fell to chance (0.550) when the pattern moved position.
57	RNN cell, forward	One cell reused at every step handles any length; information from step 0 faded to nothing by T=160.
58	BPTT	Gradient check 2.17e-08; trained from 0.395 to 1.000 at T=10.

Day	Part built	What the day's experiment showed
59	Gradient probe	Gradient reaching step 0 ranged from 1e-22 to 50,000x depending on the recurrent weights.
60	Clipping	One oversized gradient destroyed a network that had learned (0.925 to 0.463); clipping kept it (0.955).
61	Text in	Embedding and masking checked; on sentiment the RNN (0.737) lost to bag-of-words (0.817) and ignored word order.
62	All six, per word	Tagger 0.8327 against 0.8016 with no memory; order matters; clipping decisive at hidden 64.

Interview preparation — technical questions

Q: How does the backward pass of a many-to-many RNN differ from many-to-one?

A: In many-to-one the loss gradient enters only at the last hidden state. In many-to-many every timestep has its own output and loss, so at each step of BPTT you add that step's own gradient to the gradient coming back from the future: dh = dh + dH[:, t]. Everything else — undoing tanh, accumulating the weight gradients, passing dh back through Whh — is unchanged.

Q: How do you handle padding in a tagger?

A: Pad the word ids and the tag ids to a common length and build a mask. Multiply the per-word loss (and so its gradient) by the mask and divide by the number of real words. Because padding sits after the real words and contributes zero gradient, nothing flows from it into the real positions.

Q: Why reshape (batch, time, hidden) to (batch*time, hidden) before the output layer?

A: The output layer treats every word independently given its hidden state, so each word can be its own row. One matrix product and one softmax then handle all words at once, and existing 2-D helpers work unchanged.

Q: Your gradient check shows a relative error of 3e-06 on one weight. Bug or not?

A: Look at that entry. If the gradient is tiny (here 4e-07) and the absolute difference is around 1e-12, it is floating-point noise in the finite difference. Confirm by changing the nudge size: rounding error grows as the nudge shrinks, a real bug stays the same. Use a pass rule with both an absolute and a relative tolerance.

Q: How would you find out whether a sequence model is actually using context?

A: Compare with a context-free baseline on the cases that need context (ambiguous and unknown words); switch the recurrent connection off and retrain; shuffle the words at test time. Here all three agreed: gain on unknown words 0.49 against 0.37, gain gone with memory off, accuracy down from 0.83 to 0.75 when shuffled.

Q: Clipping fired on 0.6% of steps. Is it worth keeping?

A: Judge it by what happens without it, not by how often it fires. Here, at hidden size 64 and learning rate 2.0, all three unclipped runs collapsed to about 0.49 while the clipped ones reached 0.82.

Interview preparation — theoretical questions

Q: Give examples of one-to-many, many-to-one and many-to-many.

A: One-to-many: image captioning (one image, many words). Many-to-one: sentiment classification (many words, one label). Many-to-many with equal lengths: part-of-speech tagging, named-entity recognition. Many-to-many with different lengths: translation, which needs an encoder and a decoder.

Q: Why is vanishing gradient less harmful for tagging than for classifying a whole sentence?

A: Each word's hidden state receives gradient directly from its own prediction, so it does not rely on signal travelling back from the end. The recurrent path still fades (ratio 8e-04 over 20 steps here), so long-range dependencies are still hard to learn; tagging mostly needs short-range context.

Q: What is the most-frequent-tag baseline and why is it so strong?

A: Give each word the tag it had most often in training. Most words have one dominant tag, so this is right about 80–90% of the time on English. It is the minimum any tagger must beat.

Q: Why would a bidirectional RNN help here?

A: A word's tag often depends on what follows it. A left-to-right and a right-to-left tagger scored the same (0.83) but disagreed on 18% of words, and at least one was right on 90%: each direction holds information the other lacks. A bidirectional model gives every prediction both.

Q: What does the unknown-word rate tell you?

A: How much of the test input the model has never seen as a distinct word. At 20% it dominates the error: the tagger gets 97% of unambiguous known words right and 49% of unknown ones. Remedies are more data, sub-word or character features, and pre-trained embeddings.

Key takeaway and what's next

> **KEY TAKEAWAY**
> Six parts, each a few lines and each checked on its own day, combine into a working sequence model, and going from one output per sentence to one output per word costs a single line of BPTT. On real tagged text the result is 0.8327 against 0.8016 for a model with no memory. That gain was traced to its source (unknown and ambiguous words, read from the word before), its cost was measured (a little accuracy on easy words), and so was what the model cannot see (everything to its right).

- Many-to-many = an output, a loss and a gradient at every timestep; in BPTT, `dh = dh + dH[:, t]`.
- Mask the loss, not the state: padding rows get zero gradient and are divided out of the average.
- A loss at every word keeps gradient at every position (3e-02 at word 0) even though the recurrent path still fades (1e-05 for a last-word-only loss).
- Memory helped on unknown words (0.49 against 0.37) and ambiguous words (0.87 against 0.85), and cost a little on easy ones (0.969 against 0.981).
- Shuffling the words dropped accuracy from 0.83 to 0.75: unlike Day 61, this model uses order.
- Clipping was idle at hidden 32 and decisive at hidden 64. Test safety parts by removing them.
- A right-to-left copy scored the same with different errors: the case for bidirectional models.
- When a check or a prediction comes out unexpected, look at the entry before believing or dismissing it.

Suggested watch order, if starting from zero

- 1. Types of RNN | Recurrent Neural Network Types | Deep Learning Tutorial 34 (codebasics) — the shapes an RNN can take: one-to-many, many-to-one, many-to-many.
- 2. Part Of Speech POS Tagging: NLP Tutorial For Beginners - S1 E11 (codebasics) — what part-of-speech tags are and why tagging is useful.

Day 63 preview

- **The LSTM cell: gates explained**, motivated directly by Day 59's vanishing-gradient problem. Today's probe showed the recurrent path of a plain RNN passing back less than a thousandth of the gradient over 20 words. The LSTM adds a second, protected memory line with gates that decide what to keep, what to forget and what to output.
- Week 10 follows the same pattern as this week: six parts with short exercises, then a build day.